

Bandit - Level 10

Goal

The password for the next level is stored in `data.txt`, which contains **base64-encoded data**.

Connection

```
ssh -p 2220 bandit10@bandit.labs.overthewire.org
```

Use the password retrieved at the end of [Bandit - Level 09](#).

Concepts

base64 is an **encoding**, not encryption. It maps arbitrary binary data onto a 64-character alphabet (`A-Z`, `a-z`, `0-9`, `+`, `/`, with `=` as padding) so it can survive transport through channels that only tolerate text. There is **no key** — anyone holding the string can reverse it.

Recognising it on sight: only alphanumeric plus `+/=`, length a multiple of 4, and trailing `=` or `==` padding. This file ends in `==`, which is the giveaway.

```
base64 file          # ENCODE (default direction)
base64 -d file       # DECODE
```

The default direction is **encode**. That matters: running `base64` on already-encoded data doesn't fail, it just encodes it a second time and returns a longer, still-valid base64 string. No error, wrong answer.

Attempts

```
ls
# → data.txt

cat data.txt
# → VGhliHBhc3N3b3JkIGlzIHBZZk9ZNkh3VXNEajVyTDlVdnloVTdNQ212OHZONVJvCg==
# Trailing == confirms base64.

base64 data.txt
# → VkdobElIQmhjM04zYjNka0lHbHpJSEJaWms5Wk5raDNWWE5FYWpWeVREbFZkbmxvVlRr...
# ENCODED it a second time instead of decoding. No error – just a longer
# base64 string. Missing -d.

base64 -m32 data.txt
# → error: unexpected argument '-m' found
# -m32 was lifted from the SSH login banner, where it appears under "Tips"
# as a COMPILER flag (gcc -m32, compile for 32-bit) for later binary-exploitation
# levels. Nothing to do with base64.
```

Solution

```
echo "VGhlIHBhc3N3b3JkIGlzIHBZZk9ZNkh3VXNEajVyTDlVdnloVTdNQ212OHZONVJvCg==" | base64
-d
# → The password is pYf0Y6HwUsDj5rL9UvyhU7MCmv8vN5Ro
```

Simpler and less error-prone — no copy-paste of a long string:

```
base64 -d data.txt
# → The password is pYf0Y6HwUsDj5rL9UvyhU7MCmv8vN5Ro
```

`base64` accepts a filename argument directly, so piping `echo` into it is unnecessary work. Both are correct; the second scales to files that don't fit on a terminal line.

Points of Friction

1. Ran `base64` without `-d`. The command's default direction is encode, and encoding valid base64 produces more valid base64 — a **silent wrong answer** rather than an error. Same failure mode as `uniq -u` without `sort` in [Bandit - Level 08](#): plausible output that answers a different question. When a decode command returns something that still looks encoded, check the direction flag first.
2. Took `-m32` from the login banner. The banner mixes several unrelated things: the `/tmp` working-directory advice (relevant now), ASLR and compiler flags (relevant around Level 25+), and the installed tool list (`gef` , `pwndbg` , `radare2` — binary exploitation, much later). None of it is level-specific. `base64 --help` would have listed the real flags in one second.
3. Reached for a search engine before `--help`. Web search returned a correct answer, but `base64 --help` and `man base64` were faster, local, authoritative, and version-accurate. Worth building the reflex: local docs first, web second. Already in [Linux - Command Line Reference](#) under *Getting Help*, now with a reason attached.

Key Takeaway

`base64` is reversible by design and by anyone. It exists to make binary data safe to transport through text-only channels — email, JSON, HTTP headers, URLs — not to protect it. **Encoded is not encrypted**, and treating base64 as though it hides anything is a real and recurring security failure, not a beginner's misconception.

Also worth carrying forward: a tool whose default direction is the opposite of what you want will not warn you. `base64` encodes by default, and encoding encoded data succeeds. Full treatment: [Linux - Encoding vs Encryption](#).

Next

```
ssh -p 2220 bandit11@bandit.labs.overthewire.org
```

◀ Previous: [Bandit - Level 09](#) · Next ▶ [Bandit - Level 11](#)

```
Dataview (inline field '='): Error:
```

```
-- PARSING FAILED -----
```

```
> 1 | =  
    | ^
```

Expected one of the following:

'(', 'null', boolean, date, duration, file link, list ('[1, 2, 3]'), negated field, number, object ('{ a: 1, b: 2 }'), string, variable

Dataview (inline field '='): Error:

-- PARSING FAILED -----

```
> 1 | =  
    | ^
```

Expected one of the following:

'(', 'null', boolean, date, duration, file link, list ('[1, 2, 3]'), negated field, number, object ('{ a: 1, b: 2 }'), string, variable